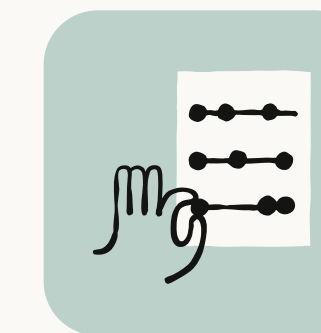


How Anthropic enables self-service data analytics with Claude

Category
Enterprise AI
Product
Claude Code
Date
June 3, 2026
Reading time
5 min
Share
Copy link

As many data science and data engineering teams can attest, enabling self-service business analytics has traditionally been a slog.

Making the data model more accessible to less technical coworkers via wide and denormalized tables often leads to overlapping views with inconsistent definitions as the business scales (and does little to bridge the gap for employees with little desire to learn SQL). Alternatively, creating more ringfenced environments for users often misses the long tail of business questions and leads to metric and dashboard bloat as teams silo their work.

The rise of LLMs provides an additional path for self-service analytics that avoids those challenges. However, pointing Claude at a warehouse and letting the agents execute can create a false sense of precision.

The initial elation of liberation from ad-hoc requests turns into dread with the realization that this setup separates stakeholders from the underlying infrastructure, documentation, and expertise that previously steered them toward carefully curated datasets.

At Anthropic, 95% of business analytics queries are automated via Claude, with ~95% accuracy in aggregate. By giving this often rote, repetitive work to Claude, our data science team can focus on more strategic work like causal modeling, forecasting, and machine learning.

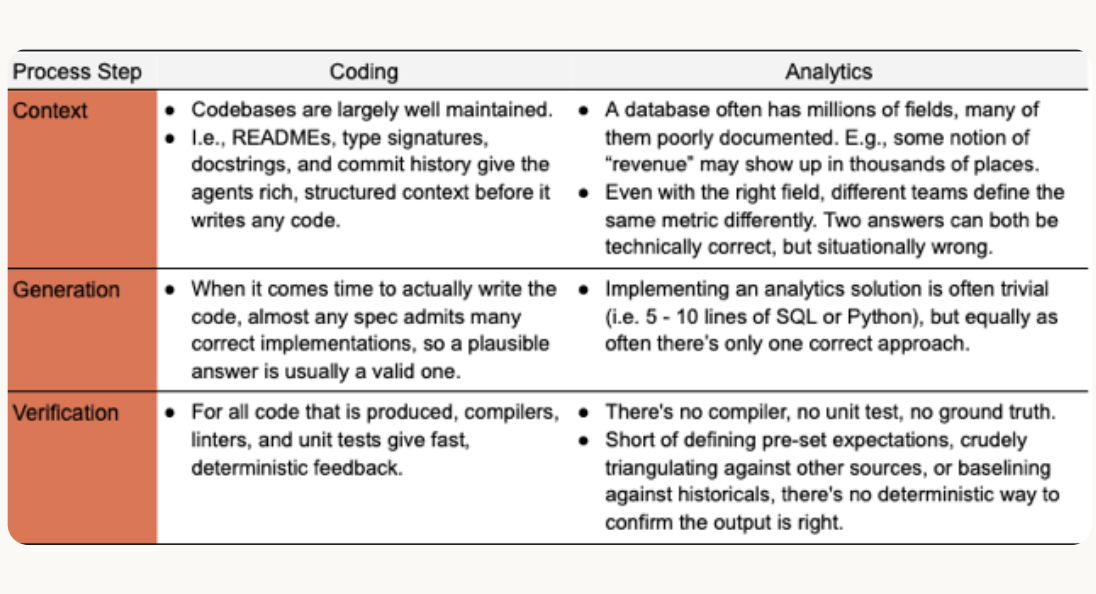
After meeting with dozens of Anthropic's top Claude Code users and having seen myriad design patterns for analytics agents, we've cultivated some best practices for other data teams working with LLMs. In this post, we'll share these tips and approaches to maximizing Claude's ability to drive self-serve business insights, including:

- Why analytics accuracy is a context and verification problem, not a code generation issue;
- The three failure modes that cause most errors;
- The agentic analytics stack we built to address these errors;
- How we measure effectiveness; and
- A basic template for how we create the majority of our skills (see the appendix)

Data is not software

LLMs' generative abilities are a double-edged sword: the mechanisms that enable creative solutions to complex problems can also hallucinate erroneous output. To fully understand the challenges with analytics agents, it's useful to compare them to coding agents.

Coding is an open-ended solution space that rewards the models' creativity, while documentation and tests provide natural guardrails against hallucination. In contrast, for analytics use cases, there's often only a single correct answer using a single correct source in which there's no deterministic way of proving the correctness.



Process Step	Coding	Analytics
Context	• Codebases are largely well maintained. • I.e., READMEs, type signatures, docstrings, and commit history give the agents rich, structured context before it writes any code.	• A database often has millions of fields, many of them poorly documented. E.g., some notion of "revenue" may show up in thousands of places. • Even with the right field, different teams define the same metric differently. Two answers can both be technically correct, but situationally wrong.
Generation	• When it comes time to actually write the code, almost any spec admits many correct implementations, so a plausible answer is usually a valid one.	• Implementing an analytics solution is often trivial (i.e. 5 - 10 lines of SQL or Python), but equally as often there's only one correct approach.
Verification	• For all code that is produced, compilers, linters, and unit tests give fast, deterministic feedback.	• There's no compiler, no unit test, no ground truth. • Short of defining pre-set expectations, crudely triangulating against other sources, or baselining against historicals, there's no deterministic way to confirm the output is right.

For self-service agentic business analytics, the complexity mainly lies in the ambiguity of the data. The central problem comes down to our ***ability to map a user's question to specific and up-to-date entities in our data model and know the correct way of working with them***. If we can do that, then the resulting execution and SQL becomes trivial.

We've identified three attributes of this problem that account for an overwhelming majority of inaccurate responses:

1. **Concept <> entity ambiguity**: with hundreds of viable options in a data model (out of potentially millions of fields), the agent is unable to choose the correct fields that best answer a user's question. For example, in measuring the number of active users: what actions constitute being "active"? Do you include fraudulent users? What lookback window do you use?

2. **Data staleness**: data sources, business definitions, and schemas change constantly; assets and agent knowledge go stale and start returning subtly wrong answers.

3. **Retrieval failure**: the right information may actually be in the data model and properly annotated, but given the vastness of the search space, the agent simply doesn't find it.

Get Claude Code `curl -fsSL https://claude.ai/insta…`

Or read the documentation

Our agentic analytics stack

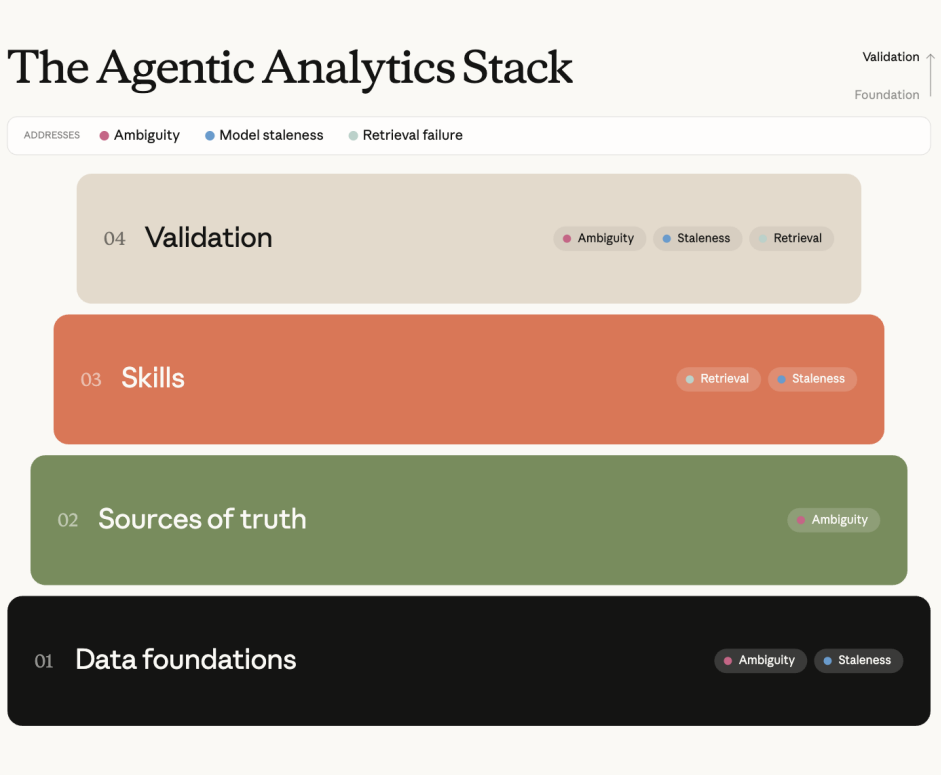
At Anthropic, the main way we minimize these three errors is via our agentic data stack. Each layer exists primarily to attack one or more of these problems:

1. **Entity ambiguity**: data foundations and sources of truth shrink the space of plausible entities until there's a single governed answer.

2. **Staleness**: maintenance and validation processes keep everything from rotting as the business changes.

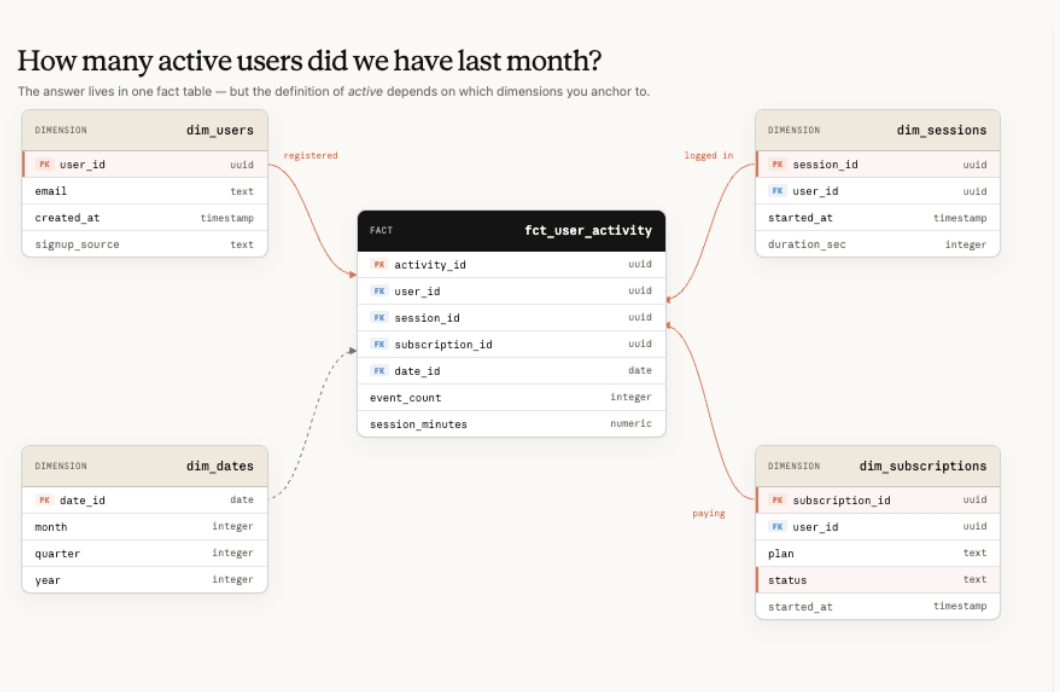
3. **Retrieval failure**: skills make sure the agent reliably finds and correctly uses that answer.

In this section, we'll discuss how we built each layer.



Data foundations

The most important aspect of ensuring analytics agents are accurate is via strong data foundations, which include the data models, transforms, tests, and tables in a data warehouse, along with the metadata describing them. Standard data engineering and data quality practices such as dimensional modeling, shift-left testing, freshness and completeness checks on critical pipelines all still apply (and we won't relitigate these).



Standard data engineering practices like dimensional modeling are just as important as they ever were.

What does change is that the end user of your data model is no longer a data expert (e.g. data scientist), but rather agents acting on behalf of users with varying degrees of data expertise or understanding of the underlying infrastructure. This shift presents a challenge in that the results can't require the user to validate the underlying correctness simply because the end user doesn't know.

The data foundations layer is aimed primarily at ambiguity: if *revenue*, for example, resolves to one governed dataset instead of forty plausible candidates, the problem largely disappears before the agent ever has to search. It's also where the first staleness defense lives, since the same repo that defines the canonical models is the natural place to enforce that they stay current.

We've seen a few practices work especially well:

- **Create canonical datasets**: By far the most common failure is that the agent can't map a concept ("revenue for product X") to the single correct table, column, and metric definition, usually because there are multiple plausible candidates with subtly different implementations. The fix is fewer, more heavily governed logical models: curate a small set of canonical, single source-of-truth datasets that are clearly owned, consumption-ready, and discoverable, then aggressively deprecate the near-duplicates. Physical rollups and caches still matter for cost and performance, but they should derive mechanically from the canonical models rather than living alongside them as alternatives. The goal is that when an agent searches for a concept, it finds a single governed answer.
- **Enforce your standards**: We've found the foundations only hold if the canonical models and metric definitions are enforced by *tooling* (the agent is structurally routed to them first; more on that below), by *CI* (changes that bypass them fail review), and by *mandate* (downstream teams build on the governed layer or explain why not). Governance without enforcement otherwise quickly decays back to the multiple candidates problem.
- **Colocate artifacts**: Our main defense against constantly changing data models and business logic is colocation. Nearly all data code (i.e., modeling, semantic layer, reference docs, canonical dashboard definitions) lives in a single repo, with CI checks that protect cross-layer integrity. If a modeling change would break a downstream dashboard or invalidate a documented metric, CI flags it and the fix ships in the same PR. (We'll come back to the mechanics of this in the **Skills** section below.)
- **Treat metadata as a first-class product**: Coding agents perform well partly because codebases are *legible*: READMEs, type signatures, docstrings, etc. Your warehouse can be just as legible, but only if column and table descriptions, canonical metric definitions, grain documentation, valid value ranges, lineage, ownership, and model tiering are maintained with the same rigor as the transformations themselves. While not a new insight, good governance provides critical context that helps the agent choose the right dataset.

Sources of truth

If data foundations are the data warehouse itself, sources of truth are the reference surfaces the agent consults to navigate it. This layer reduces concept <> entity ambiguity and turns "weekly active users" in a stakeholder's question into a specific, governed entity in your data model. Roughly in descending order of trust:

- **Semantic layer:** the compiled metric and dimension definitions. If a question maps cleanly to a defined metric, the agent calls a function and gets one number, the same number every other surface in the company produces. Our agents are *structurally required* (by skill instruction) to leverage the semantic layer first (see the appendix). One idea we tried that *didn't* work: bootstrapping the semantic layer by having an LLM auto-generate metric definitions from raw tables and query logs. It produced plausible-looking definitions that encoded the very ambiguities we were trying to eliminate, and was net-negative on our evals versus a smaller, human-curated layer. Therefore we recommend generating the *documentation* with Claude, but having a human own the *definition*.
- **Lineage and the transformation graph:** when the semantic layer doesn't cover a question, lineage and table ranking (based on number of references) let the agent reason about which upstream models feed a concept, which are deprecated, and which share grain. This transforms "I don't know the metric" into "I know which governed model to aggregate from." It's also the backbone of the freshness and provenance signals we surface in **online validation** below.
- **Query corpus:** historical SQL from dashboards, notebooks, and prior analyses. Intuitively, this should be high-value: it's a record of every question already answered correctly. *In practice, we found that giving the agent raw retrieval access to thousands of prior queries moved accuracy by less than a point* (we walk through that ablation in a later section below). Unstructured retrieval couldn't map a new question to the right precedent. What does work is distilling that corpus into structured per-domain reference docs and reusable analysis patterns described in **skills**. Treat the query history as raw material for curation, not as a source of truth the agent reads directly.
- **Business context:** the layer most teams skip, and the one we underrated the longest. An agent that doesn't understand your business will answer what the user asked, but not what they meant. It won't know that "the Q2 launch" refers to a specific product, that two teams define the same term differently, or that a question is being asked because a board meeting is on Thursday. We pipe in a company knowledge graph consisting of indexed docs, roadmaps, decision logs, and our organizational structure so the agent can resolve ambient references and ask better clarifying questions.

The common failure pattern across all four is the same one from the data foundations layer: **poor or stale documentation**. Claude is exceptionally useful for closing the gap (drafting column descriptions, proposing metric docs from query patterns, flagging undocumented models in CI), but the curation and ownership are managed by humans.

In the next two sections, we discuss how to make that ownership cheap enough that it actually happens.

Skills

If the sources of truth are the agent's *declarative* knowledge (i.e., what a metric means) then a skill is its *procedural* knowledge: which sources to consult in what order, how to navigate ambiguous data, and what a finished analysis looks like.

In Claude Code, a skill is a folder of markdown the agent reads on demand. At Anthropic, the skills we developed are hugely value additive. Without skills, Claude's ability to answer analytics questions accurately didn't exceed 21% on our evals. Adding skills gets these numbers consistently above 95% in aggregate and regularly around 99% in certain domains. See the appendix for a skeleton we use to create a majority of our skills.

Some best practices:

Create pairwise skills: a ***knowledge*** skill acts as a thin top-level router that allows additional domain details to load on demand. It says "try the semantic layer first, but if there's no coverage, here are ~30 reference files for this domain describing the relevant tables, columns, joins and gotchas." This router is, in effect, our answer to retrieval failure: rather than letting the agent search a million-field warehouse, it narrows the space to a few dozen curated files before a query is ever written. The ***unbook*** skill encodes the process a senior analyst would follow: clarify the question, find sources (via the knowledge skill), run the query, and then loop the result through adversarial review sub-agents. It also bundles a dozen reusable analysis patterns (retention curves, rate decomposition, funnel analysis) so that common requests don't get reinvented each time.

Create proper reference docs: written for retrieval by an LLM. Our reference docs describe tables (grain, scope, and exclusions), the mechanics of gotchas (e.g., "exclude known free-email domains, but keep custom ones like anthropic.com"), and explicit routing triggers (e.g., "IF the question is about experiment lift… DO NOT use for raw event counts") without prescriptive recipes that go stale. See below for a skeleton we use to create reference docs.

```
# [Domain] Tables

## Quick Reference
### Business Context — [what this domain means in plain
words]
### Entity Grain — [what one row represents]
### Standard Hygiene Filter — [the filter every query in
this domain applies]

## Dimensions
- [How the key dimensions are encoded, and how the same
concept is named
  differently across tables]

## Key Tables
### [table_name]
- **Grain**: [...] · **Scope/exclusions**: [...]
- **Usage**: [when to use it, when NOT to, join keys,
required filters]
[... one short section per governed table ...]

## Gotchas
- [The wrong-answer modes a senior analyst would warn you
about]

## Best Practices / Common Query Patterns
- [Default choices, standard cuts, worked patterns where
the exact query
  form is the hard part]

## Cross-References
- [Neighboring domain docs that own adjacent questions]
```

Treat skill maintenance as a first class citizen: Skill docs describe a data model that changes daily, so without active maintenance they're wrong within weeks. We watched our offline accuracy drift from ~95% at launch to ~65% over a month before we treated this as an engineering problem. That meant colocating skill markdown files in the same repo as our transformation models, so the PR that changes a model is the same PR that updates the doc describing it. A code-review hook flags any reporting-model change that doesn't touch a skill file. Roughly 90% of our data-model PRs now include a skill change in the same diff. We also regularly prune skill scaffolding as models improve and previous failure modes no longer apply.

Create a consistent and seamless experience across all surfaces: the same skill *must* provide the same answer to questions in Slack, in the IDE, in a dashboard tool, and in standalone agent sessions. We did this by ensuring one canonical source (the data repo) and that skill changes are synced automatically. On merge, the skill syncs to a plugin marketplace (for IDE users), to cloud-storage blobs (for hosted apps that read a single file), and is served directly as resources over MCP. We also designed for portability from the start by avoiding hardcoded repo paths and surface-specific namespaces.

Validation

Finally, validation is how you find out which of the three failure modes is still leaking through.

Offline evaluations

A common pattern we see is that data teams will set up elaborate analytic environments without having any process to understand the accuracy of their analytics agents.

One way of addressing this gap is via offline evals, which are simple question / answer pairs. You can think of offline evals similar to offline testing for an ML model in that they don't tell you the performance of your online agents, but they do give you a good sense of whether you'll have any critical gaps.

We deploy two kinds of offline evals at Anthropic. **Dashboard-based evals** are auto-generated by Claude (then human validated), covering the most common stakeholder questions. **Long tail evals** are where we feed Claude business context (roadmaps, table docs) and have it generate plausible questions across the rest of the domain. We also continuously harvest every time a stakeholder corrects the agent in a thread as that correction is a candidate eval.

Other best practices, include:

- **Anchor ground truth so it can't drift**: An eval written against live data goes stale the moment the underlying number moves. Pin every eval to a snapshot date, write it against a stable fact table, or have the grader judge the agent's *query* rather than its number. Wire the suite into CI so a PR touching a dependency re-runs the affected evals.
- **Store results like telemetry, not like test logs:** Every run lands in a warehouse table with the skill version, git SHA, model ID, per-assertion pass/fail, token count, and wall-clock. "Did that change help?" becomes a query, and you get the time-series to catch slow regressions that a single CI run won't.
- **Gate launches per domain**: A domain owner can't announce the agent to their stakeholders until their slice of the eval set clears some threshold (we initially used ~90%). It forces reference-doc fixes *before* users see the failures.
- **Create the appropriate number of evals**: The number of evals you should have depends on the complexity of the business area and the complexity of the underlying data model. Calibrate by tracking how well offline accuracy predicts online accuracy: we've found there are diminishing returns past a few dozen per topic (e.g., "growth"), and that ceiling drops with each new model generation.
- **Offline eval accuracy should be ~100%**; every correct answer should also be hitting your semantic layer (if you have one). Again, this level of accuracy doesn't tell you your system isn't going to produce a wrong answer, just that there are no obvious gaps, assuming you have proper eval coverage.

Ablation techniques

Every structural decision about the skill (e.g., which sources to expose, whether a sub-agent earns its latency, whether to merge two skills into one) is made by holding our offline eval set fixed.

We vary exactly one component and compare pass rates. Each run only takes an hour and replaces a lot of arguments. The methodology matters more than any single result:

- **Design for null results.** Our most useful ablation was a negative

one. We gave the agent direct grep access to our entire dashboard, transformation, and analyst-notebook SQL (thousands of files). We then verified in transcripts that it actually read them before every answer. Accuracy moved by less than a point in either direction. We then checked the obvious confounds: was the answer actually in the corpus for the questions it got wrong? About 80% of the time, yes. Did "answer present" predict "now gets it right"? No, the flip rate was flat. The information was there, the agent saw it, and it still didn't use it. That single experiment told us our bottleneck wasn't *access* to prior work, it was *structure* (i.e., mapping a question to the right entity). That insight redirected months of roadmap.
- **Ablate at PR granularity.** Every meaningful skill edit gets a before / after run on the relevant eval slice, with the delta in the PR description. It keeps "I improved the docs" honest and catches the surprisingly common case where a well-intentioned addition makes things worse.
- **Keep a short list of what didn't work.** Two of ours: stacking additional rounds of doc refinement past a certain point (we hit three consecutive net-negative iterations: the docs were getting longer, not better), and swapping the adversarial reviewer to a cheaper model to cut latency (it lost most of the accuracy wins, for no real speedup). Negative results are cheap to record and they prevent the next person from re-running the same experiment.

Online validation

The final step is ensuring the actual online system performance is as accurate as possible. Some of the steps we take include:

- **Adversarial review**: we've found that employing a Claude skill to aggressively challenge all underlying assumptions on a potential final answer increased accuracy by 6% within our eval set, but at the cost of 32% more tokens and 72% higher latency.
- **Provenance footer:** every response carries a footer that contains which source tier it came from (semantic layer › curated reference › raw table), how fresh the underlying data is, and who owns the model. It doesn't make the answer more correct, but it does help the consumer judge how much they can trust the response. A "raw table, freshness unknown" footer is a signal to verify before forwarding upstream, and it's one of the few mitigations we have for silent failures.
- **Data quality checks**: it's possible that your agent is using the right field in the appropriate way, but the data itself is incorrect. Adding basic data quality checks to ensure the referenced field is up-to-date, complete, and has no anomalies is generally good hygiene.
- **Passive monitoring:** two production signals we track continuously are the share of agent queries that resolve through the semantic layer, and the share of responses that use correction language ("that's the wrong table," "you're missing the fraud filter"). Both feed a dashboard reviewed weekly alongside the offline pass rate.
- **Active correction harvesting**: the part that closes the loop. A scheduled agent scans stakeholder channels every few hours for similar correction language, drafts a one-line fix to the relevant reference doc, and opens a PR tagged to the domain owner. The fix path is deliberately boring — edit a markdown file, merge, auto-sync everywhere — so a domain owner doesn't spend too much time on the task. The same corrections feed back into the offline eval set.

The failure mode none of this fully catches is the **silent** one. The answer is wrong, but looks plausible and is used without objection. Our mitigations are the provenance footer, explicit human sign-off on anything leadership-bound, and a standing eval for each domain's top KPIs that sanity-checks against the blessed dashboard daily, though we don't have a robust solution yet.

Getting started

If you're starting from zero, a handful of canonical datasets, a few dozen offline evals, and a thin knowledge skill will capture most of the upside; everything else in this post is what we added once those were built.

We also shared many best practices, and not all of them will be appropriate for every data team. Align with your organization on a few principles that will affect your approach by asking:

- **How important is a correct answer today vs. in the future**? AI models are progressing at a rapid pace. We often see companies building a significant amount of infrastructure to account for current model shortfalls that become moot once those models improve. Knowing where models fall short, and waiting for model improvements to fill the gap has significantly less overhead, but may not fit your company's risk tolerance.
- **How do you anticipate the complexity of your business to change over time**? Some of the processes we discussed may be overkill if, for example, you don't produce much data, you only have a few consumers of the output, or your data model is likely to remain simple.
- **How technical is the intended audience of the output**? Phrased differently, if you're building this analytics system for data scientists who can recognize when an answer is incorrect, you may be more tolerant of errors compared to a situation in which the audience has no familiarity with the underlying data model.
- **How much are you willing to spend for improved accuracy**? We've found certain processes like adversarial validation can significantly improve accuracy, but often at a higher cost and latency.
- **What is your comfort around access controls and internal data privacy**? Agents are often significantly more performant the more context they have; however, broad data access cuts against most companies' governance posture. This determines whether you're building one agent or many scoped ones.

Whatever your route, our greatest gains have come from addressing each of the three failure modes: collapsing ambiguity into a single governed answer, making the answer easily discoverable, and flagging when either has gone stale.

This article was written by Chen Chang, Clement Peng, Justin Leder, Johanne Jiao, and Josh Cherry, members of the Data Science and Data Engineering team. The authors would like to thank Michael Segner for his contributions.

Appendix

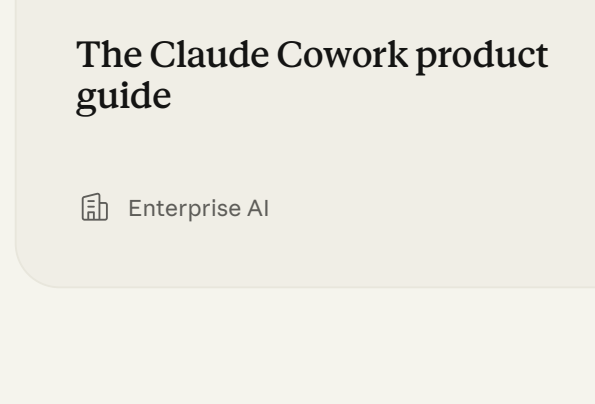
Skill File Skeleton

What follows is the skeleton of our main warehouse skill: the real file's structure, with internal specifics replaced by [bracketed placeholders]. It isn't meant to be copied verbatim; it's meant to show the kinds of sections we found worth writing down.

```
---
name: [warehouse-skill]
version: [x.y.z]
description: "IF the user asks to query [the company]'s
data warehouse for any
  [list of business domains] question — THEN invoke this
skill. DO NOT invoke
  for [adjacent engineering tasks] or questions with no
data-warehouse component."
---

# [Warehouse] Skill Instructions

## Description
The single source of truth for safe and effective
[warehouse] querying.
Referenced by other skills [listed] for query execution
guidance.

Act as a Data Analyst, providing strategic insights and
data-driven
recommendations but seek guidance along the way.

**Out-of-scope decisions**: [product areas, etc.] → surface
data only,
state "decision is [owning team]'s call", do NOT take a
position or author
code fixes.

## Executing queries
Priority:
1. **[Managed connection]** (if available): [query tool] /
[schema tool]
2. **[CLI fallback]** (if installed): [default project,
fallback project]
3. **Neither** — ask the user to authenticate, then stop

---

# Semantic Layer (REQUIRED first step)

The governed semantic layer is the **mandatory default
path** for every data
question — same numbers as [the BI tool],
joins/grain/filters baked in. Raw SQL
via the reference docs below is the **fallback**, used only
after the
semantic-layer path is shown not to cover the ask.

## Required workflow
1. **Load** — [how to load the semantic layer in each
runtime, with fallbacks]
2. **Discover** — search measures/dimensions by keyword;
**always check
   segments** (the named canonical population filters —
hand-rolled WHERE
   clauses for these are the dominant wrong-answer mode)
3. **Compile + run** — build the spec → compile to SQL →
execute
4. **Fallback** — only if discovery finds no relevant
metric or compile fails
   → raw SQL via `references/*.md` (PART 3 below)

> **Don't bail early.** Do NOT fall back to raw SQL on
these grounds:
> - "[custom date filtering / cohorts]" → [covered by time-
dimension specs]
> - "[needs a join]" → [the metric layer already
encapsulates its joins]
> - [3–4 more pre-rebutted excuses agents use to skip the
semantic layer]

### Date windows & timezone — decide before you query
- **As-of date vs trailing-N days**: [convention for each]
- **"Last week/month"** → the last *complete* calendar
week/month, not trailing-7/30
- **Timezone default**: [TZ]; [exception for certain
reporting rollups]
- **Freshness lag**: [some] tables settle late — anchor on
MAX(date), not "yesterday"

---

# PART 1: MUST KNOW (Read First for Every Request)

## 🚀 Quick Start Workflow
1. **Check for red flags first**: [restricted/PII requests,
gated domains,
   high-stakes asks that need extra validation]
2. **Out of scope — escalate, don't guess**: [access
requests, pipeline
   troubleshooting, stale dashboards, root-cause
assertions, product/pricing
   recommendations] → redirect to [the owning team], don't
answer
3. **Clarify the request**: time period, segment, the
business decision it informs
4. **Check for existing dashboards**: [per-domain dashboard
catalogs]
5. **Identify the data source**: [navigation map below;
prefer governed/aggregated tables]
6. **Execute the analysis**: [required filters +
adversarial review]
7. **Deliver insights**: show methodology, differentiate
observations from interpretations

## 🏢 Business Context

### Entity Disambiguation (MUST CLARIFY)
- **"[Term A]" can mean**: [entity 1] or [entity 2] —
always clarify which
- **"[Term B]" can mean**: [entity 1] → [entity 2] →
[entity 3] (one-to-many chain)
- **"Users"**: [which identifier gives accurate counts, and
which ones inflate them]

### Business Terminology
- [Current product names vs deprecated aliases that still
appear as frozen
  values in the data layer — write with the new names,
filter with the old]
- [Key internal acronyms]
- **[Headline metric] calculations**: [monthly / default
window / leading indicator]
- **Unfamiliar terms — search [internal docs], don't
guess**

### Data Integrity Requirements ⚠️
- **NEVER**: make up data/columns; make speculative
assertions beyond what data shows
- **ALWAYS**: use safe division; differentiate observations
("data shows X")
  from interpretations ("this suggests Y"); flag
limitations

---

# PART 2: HOW TO DO (Follow During Execution)

## 🔧 Technical Execution Guide
- [Managed-connection tools and CLI invocation details]
- **PII protection**: for restricted data, return the SQL
for the user to run
  themselves — do not return results

## 📊 Analysis Best Practices Guide
1. Clarify the ask before querying
2. Show your work (filters, inclusions/exclusions,
freshness)
3. Clarify denominators
4. Consider sample bias
5. Connect to business impact
6. **Adversarial SQL review (MANDATORY)** — spawn the [sql-
reviewer] sub-agent
   for every query before the final answer; blocking
findings must be fixed
   and re-reviewed; do not self-certify
7. **Report with provenance** — every answer ends with a
footer:
   > **Source:** [semantic layer | governed table | raw
exploration] ·
   > **Confidence:** [tier] · **Reviewed:** [reviewer ✓,
round N] ·
   > **Freshness:** [max date in the data] · **Owner:**
[owning team]

---

# PART 3: DATA REFERENCES & RESOURCES

## 📚 Knowledge Base Navigation
### [Domain A] → `references/[domain_a].md`
- **Use for**: [kinds of questions]
- **Key tables**: [...]
- **Dashboards**: `references/[domain_a]_dashboards.json`

### [Domain B] → `references/[domain_b].md`
- **Use for**: [...]

[... one entry per business domain — a few dozen in total
...]

## ⚠️ Troubleshooting Guide

### When Information Is Missing
- [missing tables / access denied / outdated docs / unknown
enum values → what to do]

### Field Naming Gotchas
- Use `[field_x_v2]` NOT `[field_x]`
- [Two similarly-named tables report the same metric at
different grains — which to use]
- [Which of two plausible sources is canonical for the
headline metric]
- [… a dozen more hard-won one-liners …]
```

Related posts

Explore more product news and best practices for teams building with Claude.



Jun 5, 2026

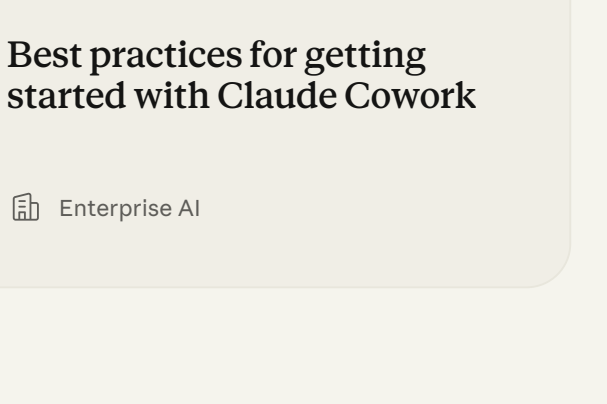
The Claude Cowork product guide

Enterprise AI

Jun 5, 2026

How one Anthropic seller rebuilt his team's workflows with Claude Code

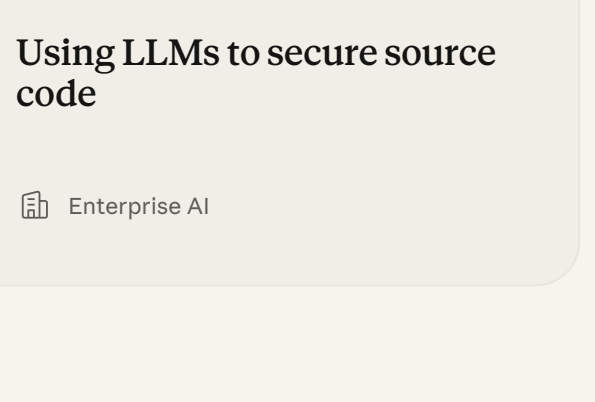
Claude Code



Jun 3, 2026

Best practices for getting started with Claude Cowork

Enterprise AI



May 27, 2026

Using LLMs to secure source code

Enterprise AI

Transform how your organization operates with Claude

See pricing
Contact sales

Get the developer newsletter

Product updates, how-tos, community spotlights, and more. Delivered monthly to your inbox.

Enter your email

Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.

Claude

How can I help you today?

Write
Learn
Code

Products
Claude
Claude Code
Claude Code for Enterprise
Claude Cowork
Claude Security
Download app
Pricing
Log in

Features
Claude for Chrome
Claude for Slack
Claude for Microsoft 365
Skills

Models
Mythos Preview
Opus
Sonnet
Haiku

Solutions
AI agents
Code modernization
Coding
Customer support
Education
Enterprise
Financial services
Government
Healthcare
Legal
Life sciences
Nonprofits
Security
Small business
Startups

Claude Platform
Overview
Developer docs
Pricing
Marketplace
Claude on AWS
Google Cloud
Microsoft Foundry
Regional compliance
Console login

Resources
Blog
Claude partner network
Community
Connectors
Courses
Customer stories
Engineering at Anthropic
Events
Plugins
Powered by Claude
Service partners
Tutorials
Use cases

Company
Anthropic
Careers
Economic Futures
Research
News
Responsible Scaling Policy
Security and compliance
Transparency

Help and security
Availability
Status
Support center

Terms and policies
Privacy choices
Privacy policy
Responsible disclosure policy
Terms of service: Commercial
Terms of service: Consumer
Usage policy

BY ANTHROP\C

© 2026 ANTHROPIC PBC